OTUS

Kotlin Backend Developer.Professional

otus.ru

REC **Проверить, идет ли запись**

Меня хорошо видно && слышно?

Разработка бессерверного приложения

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
OTUS-Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Выбор проекта	Маркетинг аналитика	Security	Архитек-тура	DevOps	
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Защита					

Окатов Сергей
автор программы, руководитель курса

- кандидат физ.-мат. наук
- 15 лет научной работы в университетах РФ, Европы, США
- 10 лет комм. разработки в стартапах и корпорациях (Сбер, Ланит) в качестве руководителя разработки, архитектора
- Специализация
  - Котлин
  - Большие данные
  - Распределенные системы
  - Архитектура

План полета

- Особенности Serverless
- Обзор SL платформ
- Создание приложения
- Деплой
- Рефлексия

Цели вебинара

После занятия вы сможете

1. Знать особенности serverless технологий
2. Принимать решение о разработке бессерверных систем
3. Выполнить деплоймент

Смысл

Зачем вам это уметь

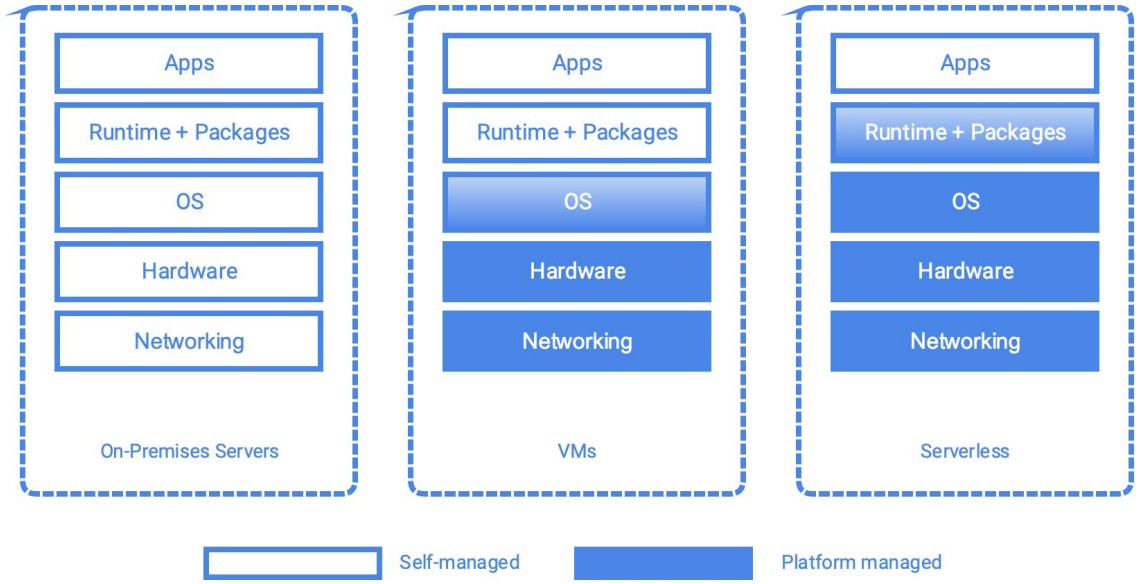
1. Бессерверные технологии - это перспективный следующий шаг в развитии архитектуры и активно применяются на Западе
2. Применение бесперверных технологий позволяет добиться существенной экономии на инфраструктуре в некоторых случаях
3.

Особенности Serverless

Бессерверная архитектура

Особенности:

- Реализация в виде функций – FaaS
- Активация по различным триггерам
- Запускаются только по необходимости
- Отсутствие администрирования компонентов (маркетинговая декларация)
- Неограниченное масштабирование при пиковых нагрузках (нужна осторожность при атаках)
- Должен быть stateless
- Важно время старта
- Низкое время старта у JS, Python, Go
- JVM стартует медленно
- Альтернатива GraalVM, Ktor Server Native

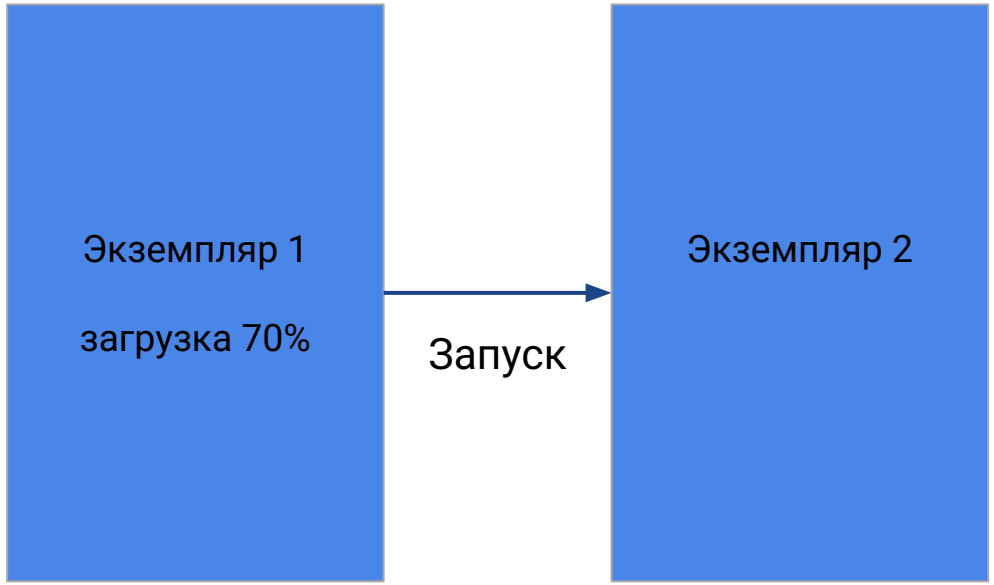


Наиболее эффективны:

- Стартапы
- Волатильная нагрузка

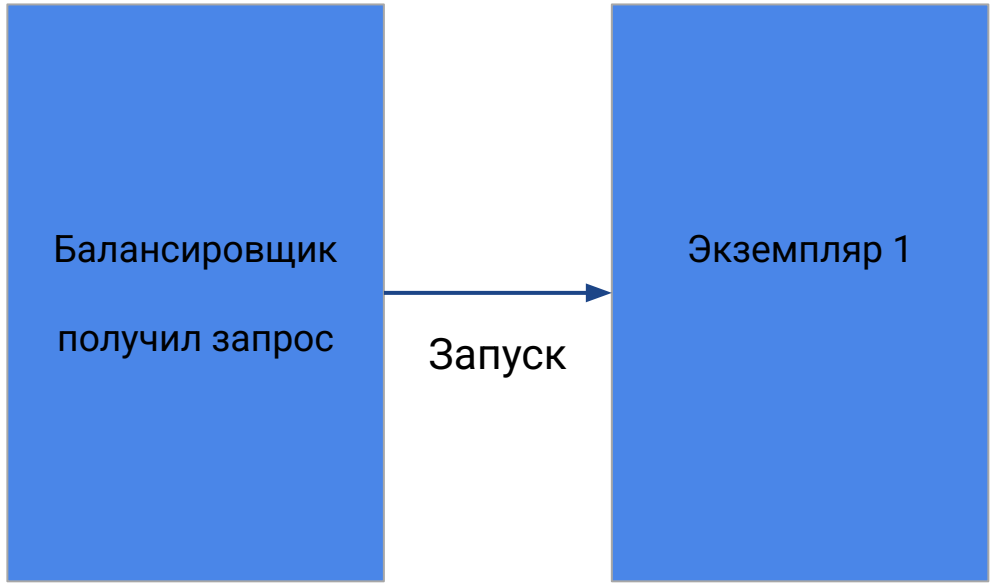
Схемы масштабирования

Kubernetes



Масштабирование на основе нагрузки

Serverless



Масштабирование на основе запросов

Провайдеры FaaS

Провайдеры:

1. Amazon Web Services
2. Google Cloud
3. Microsoft Azure
4. IBM Cloud
5. Alibaba Cloud
6. Oracle Cloud
7. Yandex Cloud

Kubernetes:

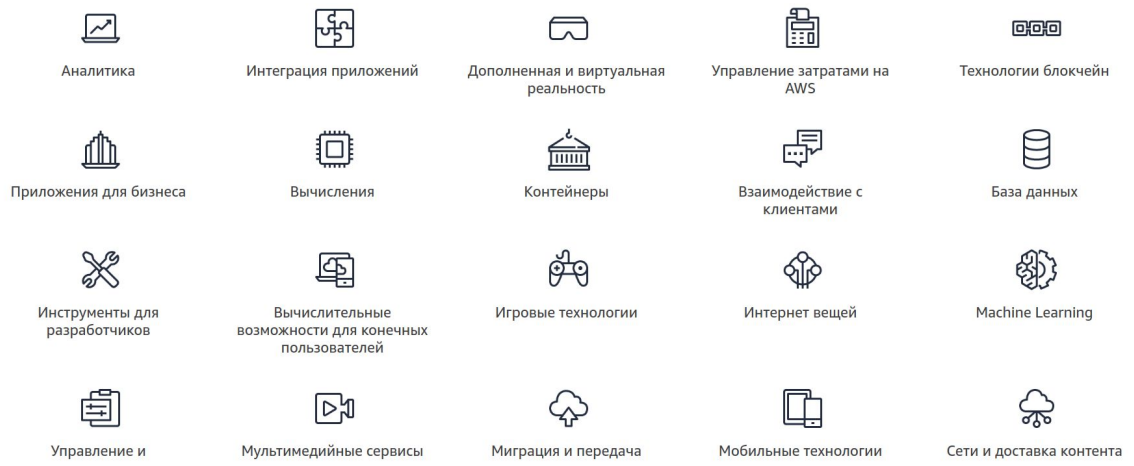
1. Kubeless
2. KNative

Краткий обзор SL platform

Устройство AWS

Наиболее важные службы:

1. API
   a. Route53 – управление доменами
   b. CloudFront
   c. API Gateway
   d. Certificate Manager
2. Хранение
   a. S3
3. Базы данных
   a. DynamoDB
   b. RDS (MySQL, Postgre, Oracle, MS SQL)
   c. ElastiCache
4. Функции
   a. Lambda
5. Логи и мониторинг
   a. CloudWatch



Недостатки:

➔ сложная структура,

➔ требует высокой подготовки

Устройство AWS: Регионы

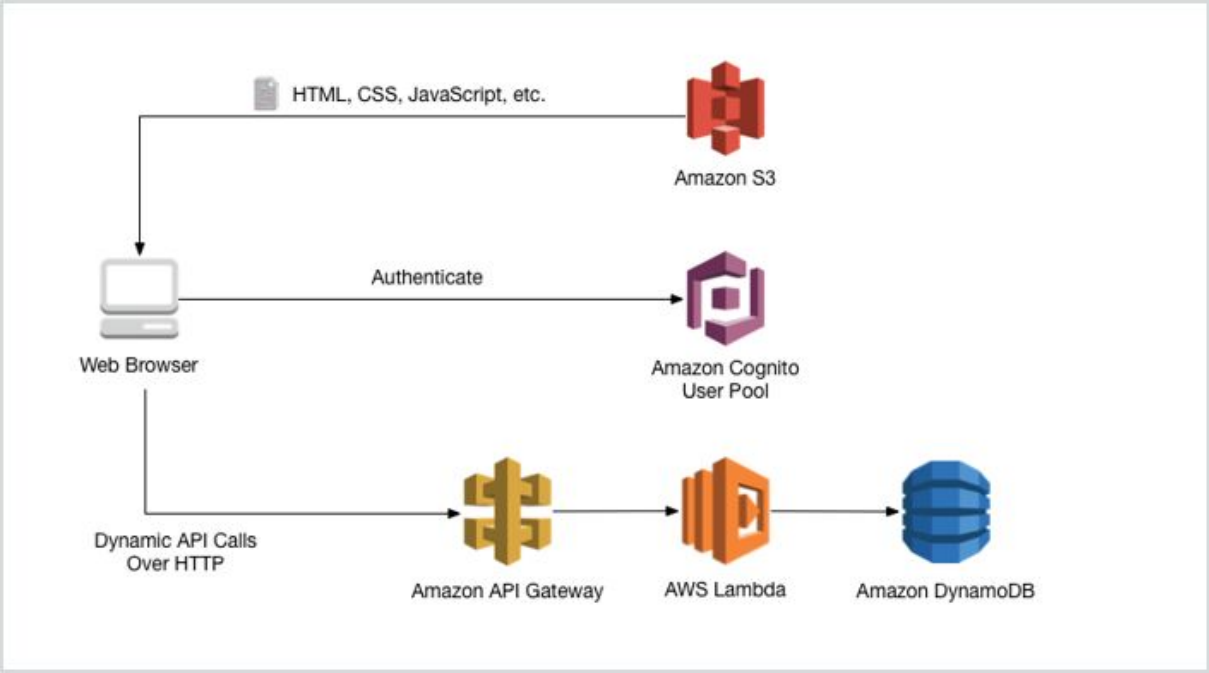
О регионах:

1. Обеспечивают гео-балансировку
2. Снижают время отклика
3. Разные стоимости услуг
4. Разный набор услуг
5. Наиболее дешевый и продвинутый us-east-1 (N. Virginia, https://aws.amazon.com/ru/about-aws/global-infrastructure/regional-product-services/)
6. Отдельная услуга для некоторых служб - синхронизация данных между регионами
7. Ни одного региона в РФ

Типичное приложение на AWS

Элементы:

1. Cognito – аутентификация
2. S3 содержит статический контент
3. API Gateway публикует REST-методы
4. Lambda – наша логика
5. DynamoDB – база данных
6. ECS - Elastic Container Service



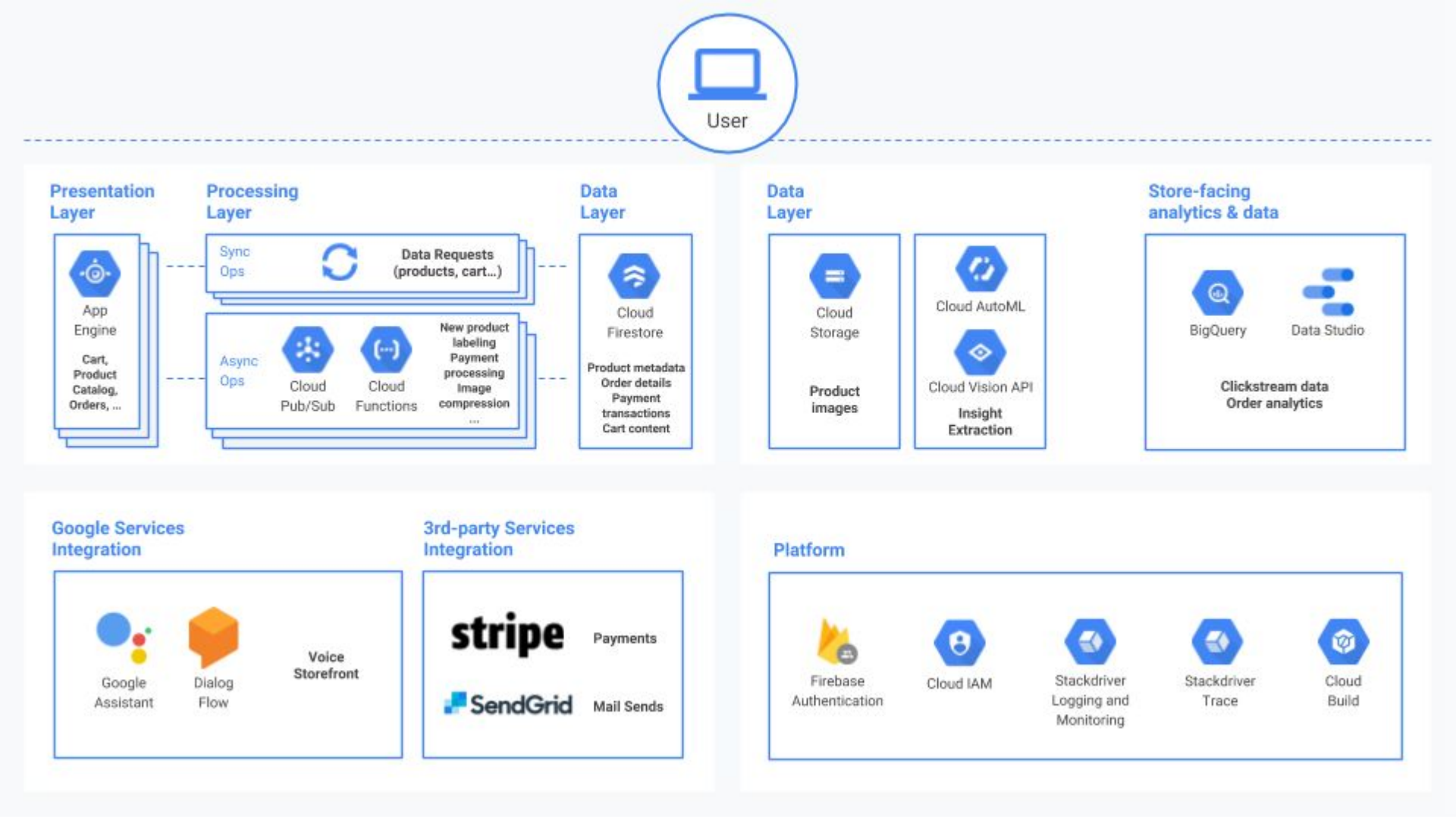
Много встроенных компонентов

AWS market для дополнительных элементов

Устройство Google Cloud Platform

Особенности:

- использует Kubernetes
- Serverless на KNative



Устройство KNative

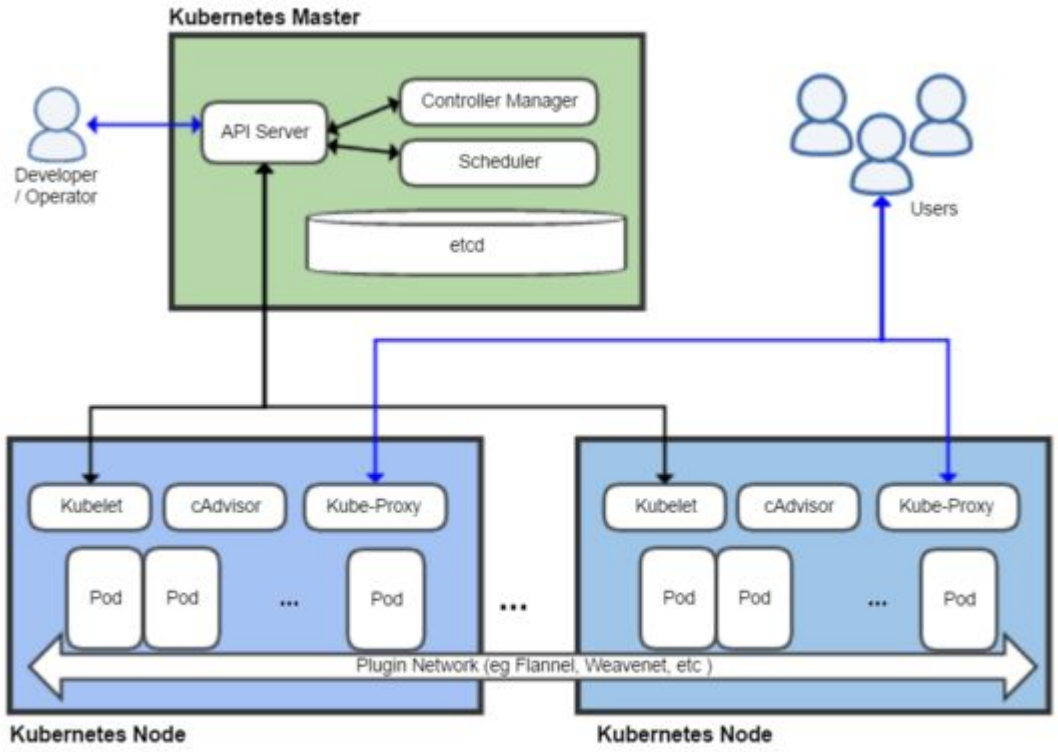
Kubernetes

Система управления кластерами:

1. Система управления
2. Узлы кластера
3. Clusters
4. Projects
5. Namespaces
6. Pods

Дистрибутивы:

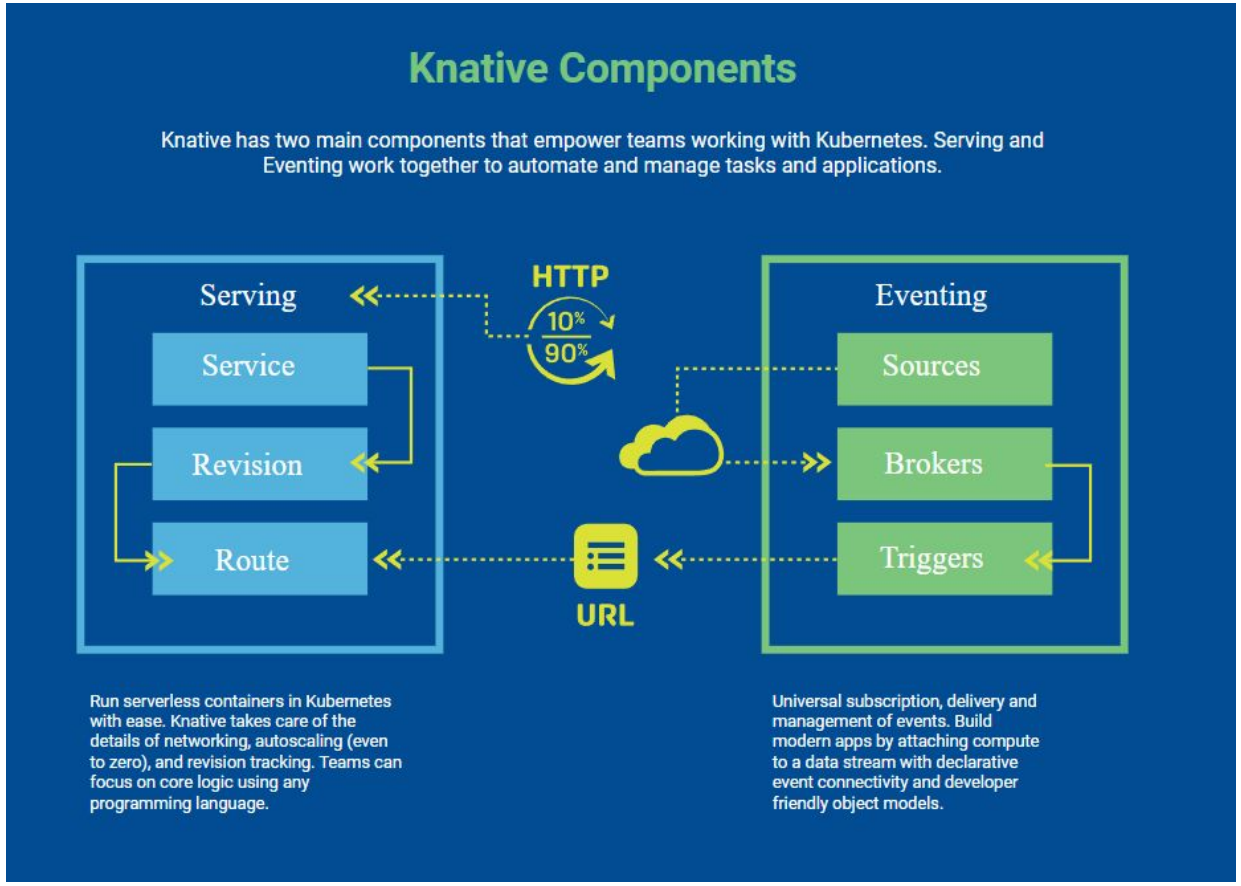
1. K3s, RKE, RKE2 - Rancher - UI
2. OpenShift, OKD - UI
3. Minikube
4. K0s
5. MicroK8s



KNative

Элементы:

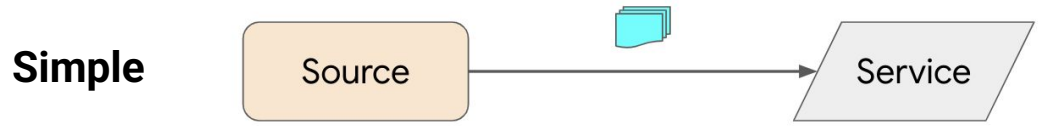
1. Разворачивается в Kubernetes
2. Публикует endpoints
3. При обращении к endpoint запускает Pod
4. Serving – обслуживание (масштабирование, деплой, запуск и пр.)
5. Eventing – обработка, доставка, управление событиями
6. Функции в виде докер-образов

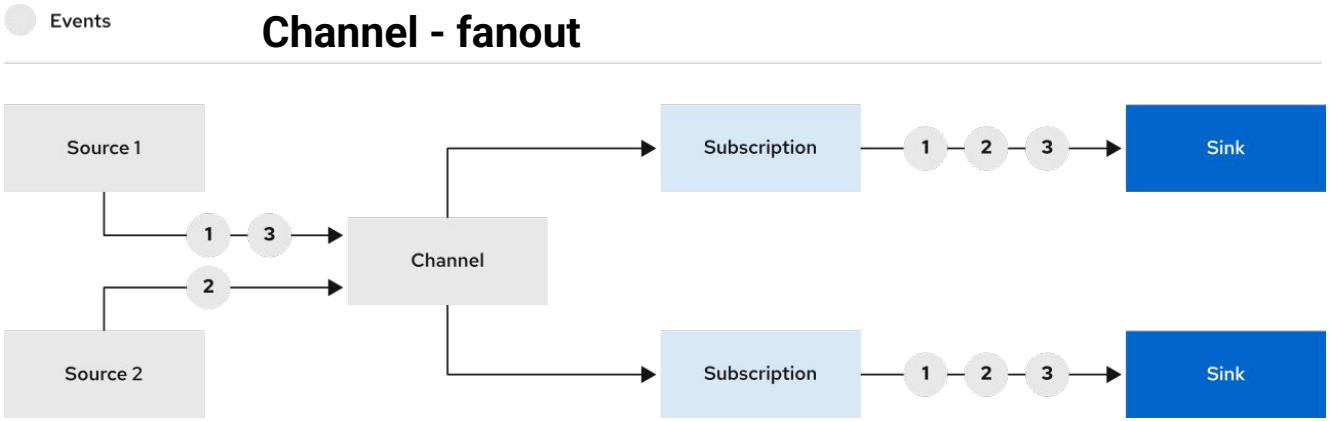


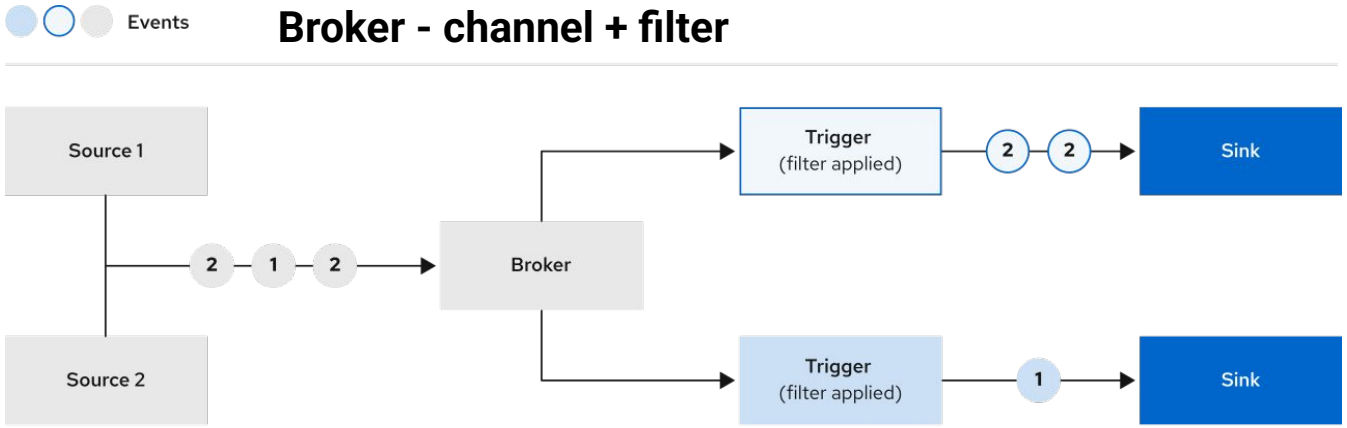
Knative

Модели распространения событий через
- Простая
- Брокер
- Канал

Элементы:
1. Event producer/source – источник событий
2. Broker – брокер сообщений (kafka, amqp)
3. Channel – канал событий
4. Trigger – заявка на подписку
5. Subscriber – подписчик на сообщений
6. Sink – сток сообщений







Яндекс Облако

Яндекс Облако

Yandex Cloud

Поиск Россия Связаться с нами Подключиться

Сервисы Решения Почему Yandex Cloud Ресурсы Тарифы Документация Блог

Облачные вычисления и разработка приложений без создания виртуальных машин.

Cloud Functions
Запуск вашего кода в виде функции

Managed Service for YDB
Распределённая отказоустойчивая Distributed SQL СУБД

Message Queue
Очереди для обмена сообщениями между приложениями

API Gateway
Интеграция с сервисами Yandex Cloud при помощи API

Object Storage
Масштабируемое хранилище данных

IoT Core
Решения для интернета вещей

Группы сервисов

- Популярные
- Инфраструктура и сеть
- Платформа данных
- Контейнерная разработка
- Инструменты разработчика
- Бессерверные вычисления
- Безопасность
- Ресурсы и управление
- Машинное обучение
- Бизнес-инструменты

Деплой

Фреймворк serverless

Особенности:

1. Автоматизирует деплоймент
2. Поддерживает различные FaaS-провайдеры
3. Поддерживает различные триггеры
4. Управляет ресурсами облака
5. Дополнительные плагины

Фреймворк Terraform/OpenTofu

Особенности:

1. Автоматизирует деплоймент
2. Поддерживает различные FaaS-провайдеры
3. Тонкие настройки каждого облачного провайдера
4. Управляет ресурсами облака
5. Дополнительные плагины

HashiCorp Terraform

OpenTofu

LIVE

Рефлексия

Рефлексия

Насколько тема была для вас сложной?

Какая тема осталась не до конца понятой?

Вопросы

1. Какие особенности у бессерверной архитектуры?
2. Каких облачных провайдеров помните
3. Как происходит деплой?

15 мин

Основные тезисы

1. Serverless является stateless и масштабируемым набором служб
2. В мире много провайдеро самый известный и мощный – Amazon Web Services
3. AWS SDK – предоставляет классы и зависимости необходимые для разработки
4. Фреймворки Serverless и Terraform автоматизируют деплой lambda-функций
5. Деплой в KNative: Kubernetes -> KNative -> Сервис
6. В YC удобнее всего поднимать функции через Serverless Containers

Домашнее задание

1. Выбрать фреймворк для выпускного проекта и реализовать приложение на нем
2. Подключить транспортные модели к приложению
3. Реализовать автоматические тесты

Срок: 09.09.2025

Цели вебинара

Проверка достижения целей

1. Знать особенности serverless технологий
2. Принимать решение о разработке бессерверных систем
3. Выполнить деплоймент

Следующий вебинар

09.09.2025

Модуль 6: Бизнес-логика проекта

Основы бизнес-логики и разработка библиотеки для шаблона CoR

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Список материалов для изучения

- https://kotlinlang.org/
- https://gradle.org/
- https://github.com/otuskotlin/202505-ok-marketplace
- https://aws.amazon.com/ru/
- https://www.serverless.com/
- https://www.terraform.io/
- https://opentofu.org/
- https://habr.com/ru/company/timeweb/blog/535586/
- https://github.com/JetBrains/kotless
- https://cloud.yandex.ru/

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание! Приходите на следующие вебинары